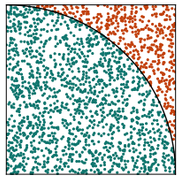
Calcular π con puntos al azar y repartir el trabajo entre 10 núcleos

Les cuento cómo estimé π con el método de Monte Carlo y cómo lo aceleré con OpenMP en mi propio portátil.

1 Lancé puntos

El método: π a partir del azar

Conté cuántos puntos cayeron dentro del cuarto de círculo, dividí entre el total y multipliqué por 4.



π ≈ 4 × dentro ÷ total

4 × 78.541.825 ÷ 100.000.000
= 3,141673

Monte Carlo: usar el azar para calcular. Error con 10^8 puntos: 0,00008

PALABRAS CLAVE

Monte Carlo · π/4 · error 1/√N

2 Repartí el trabajo

La mejora: varios núcleos a la vez

Antes: 1 núcleo, **0,545 s**

Ningún punto depende de otro, así que los partí en 400 bloques de 250.000.

```
#pragma omp parallel for
reparte los bloques
reduction(+:dentro)
suma lo que contó cada hilo
```

Cada bloque usa su propio tramo de números aleatorios.

Semilla: **2211853**

PALABRAS CLAVE

OpenMP · parallel for · reduction

3 Comprobé

La prueba: ¿siempre el mismo resultado?

Hilos	Dentro	Tiempo
1	78.541.825	0,545 s
2	78.541.825	0,276 s
4	78.541.825	0,143 s
8	78.541.825	0,079 s
10	78.541.825	0,067 s

Siempre **78.541.825**, en 63 corridas. Solo cambió el tiempo.

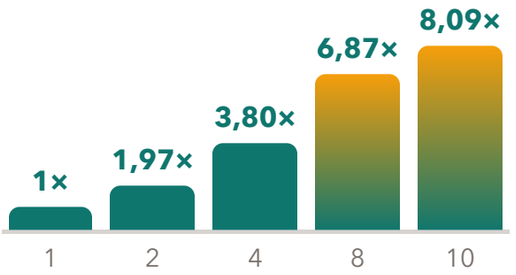
Sin reduction los hilos se pisaron el contador y con 8 hilos π dio **0,39**.

PALABRAS CLAVE

reproducible · condición de carrera

4 Medí la velocidad

La ganancia: ¿cuántas veces más rápido, según los hilos?



Pasé de 0,545 s a 0,067 s. Hasta 4 hilos va casi ideal. Desde el quinto entran los núcleos de eficiencia, más lentos: cada uno suma cerca de 0,7 al speedup.

Ley de Amdahl: f = 0,983. Lo que no se reparte pone un techo, y más hilos ayudan cada vez menos.

PALABRAS CLAVE

speedup · Ley de Amdahl

Conclusión: más hilos → más rápido | más puntos → más preciso

Sebastian Sandoval · Código 2211853 · Computación Paralela, UIS · Medido en mi MacBook Air M5 con clang y OpenMP · 4 núcleos Super y 6 de eficiencia